



Artificial Intelligence Foundations (CIS4049-N)

Week 3- Lab Introduction

Module Leader: Dr Alessandro Di Stefano

Email: A.DiStefano@tees.ac.uk

School of Computing, Engineering & Digital Technologies

tees.ac.uk/computing

Lab Session - Introduction

- NumPy
- Pandas
- Implementing agents and environments



Credit: Image by [Geralt](#) at [Pixabay](#)

Working with Data - NumPy

- Python key packages: **NumPy** and **Pandas**
- **NumPy** - fundamental package for scientific computing in Python. It contains functionality for **multidimensional arrays**, high-level mathematical **functions**
- **Pandas** is a Python library for data wrangling and analysis. It is built around a data structure called the **DataFrame** that is modeled after the R DataFrame
 - A **Pandas DataFrame** is a table, similar to an Excel spreadsheet
 - Pandas provides a great range of methods to modify and operate on this table; in particular, it allows SQL-like queries and joins of tables

Working with Data – SciPy and Matplotlib

- Other commonly used packages: **SciPy** and **Matplotlib**
- **SciPy** - a collection of **functions** for **scientific computing** in Python
 - Advanced linear algebra routines, mathematical function optimization, signal processing, special mathematical functions, and statistical distributions
 - **Scikit-learn** draws from SciPy's collection of functions for implementing its algorithms
- **Matplotlib** is the primary **scientific plotting** library in Python
 - High-quality **visualisations** (line charts, histograms, scatter plots, etc.)
 - When working inside the **Jupyter Notebook**, you can show figures directly in the browser by using the **`%matplotlib notebook`** and **`%matplotlib`** inline commands.

NumPy

- ***Ndarray*** object: encapsulates multidimensional arrays of homogeneous data types. Many ndarray operations are performed in compiled code in order to improve performance
- Main differences between **NumPy arrays** and the **standard Python sequences or lists**:
 - **Fixed size**, whereas Python lists can expand dynamically
 - **Homogeneous**: same data type and same size in memory
 - **More efficient** execution (less code) of various types of operations on large numbers of data
 - Many scientific **Python-based packages rely on NumPy arrays!!**

```
arr1 = np.array([1,2,3,4,5]) (array declaration)
```

```
arr1[0], arr1[4] (access the array elements)
```

NumPy array - Examples

```
import numpy as np
list1 = [1,2,3,4,5]
print(list1)
arr1 = np.array([1,2,3,4,5])
print(arr1)
list2 = [(1,2,3),(4,5,6)]
print(list2)
arr2 = np.array([(1,2,3),(4,5,6)])
print(arr2)
```



```
[1, 2, 3, 4, 5]
[1 2 3 4 5]
[(1, 2, 3), (4, 5, 6)]
[[1 2 3]
 [4 5 6]]
```

```
import numpy as np
list = [1,2,3]
arr1 = np.array([1,2,3])
for e in list:
    print(e)
for e in arr1:
    print(e)
```



```
1
2
3
1
2
3
```

Pandas

- **Data Frame** - a **two-dimensional data structure**
- DataFrames can be **labelled** (**rows** as well as **columns**), and the **columns can contain different data types**
- The source of the dataset can be a data file, database tables, Web service, etc.
- **Pandas DataFrame features** include:
 - Dataframe methods
 - Dataframe statistics
 - Grouping, pivoting, and reshaping
 - Dealing with missing data
 - Joining dataframes

Pandas

- Workflow with the following **steps performed with a Pandas DataFrame**:
 - **read data** into a dataframe
 - display **top** of dataframe; display **column data types**; display **non_missing values**
 - **replace NA** with a value
 - **iterate** through the **columns** and derive **statistics** for each column
 - find **missing values**; total missing values; percentage of missing values
 - **sort table values**
 - print **summary information**
 - **columns with > 50% missing**
 - **rename columns**

Pandas Dataframes - Examples

```
import numpy
import pandas
myarray = numpy.array([[10,30,20],
[50,40,60],[1000,2000,3000]])
rownames = ['apples', 'oranges', 'beer']
colnames = ['January', 'February', 'March']
mydf = pandas.DataFrame(myarray, index=rownames, columns=colnames)
print(mydf)
print(mydf.describe())
```



	January	February	March
apples	10	30	20
oranges	50	40	60
beer	1000	2000	3000
count	3.000000	3.000000	3.000000
mean	353.333333	690.000000	1026.666667
std	560.386771	1134.504297	1709.073823
min	10.000000	30.000000	20.000000
25%	30.000000	35.000000	40.000000
50%	50.000000	40.000000	60.000000
75%	525.000000	1020.000000	1530.000000
max	1000.000000	2000.000000	3000.000000

```
import pandas as pd
names = pd.Series(['SF', 'San Jose', 'Sacramento'])
sizes = pd.Series([852469, 1015785, 485199])
df = pd.DataFrame({ 'Cities': names, 'Size': sizes })
df = pd.DataFrame({ 'City name': names, 'sizes': sizes })
print(df)
```



	City name	sizes
0	SF	852469
1	San Jose	1015785
2	Sacramento	485199

Pandas Dataframes - Examples

```
import pandas as pd
df = pd.read_csv("forecast_data.csv")
print(df)
print(df.shape) # rows, columns
print(df.head()) # df.head(3)
print(df.tail())
print(df['Temperature'])
print(df['Chance Precipitation (%)'])
```



```
[5 rows x 17 columns]
0 Middle 0 48.1 Rain, Overcast
1 Middle 1 50.0 Rain, Overcast
2 Middle 2 40.3 Rain, Overcast
3 Middle 3 39.1 Overcast
4 Middle 4 33.4 artially cloudy
5 Middle 5 30.9 Snow
6 Middle 6 32.7 artially cloudy
7 Middle 7 31.4 Snow, Overcast
8 Middle 8 37.7 artially cloudy
9 Middle 9 37.9 Overcast
10 Middle 10 46.7 Overcast
11 Middle 11 40.5 artially cloudy
12 Middle 12 37.2 Overcast
13 Middle 13 38.7 Overcast
14 Middle 14 38.6 Overcast
15 Middle 15 37.4 Overcast
Name: Temperature, dtype: float64
[16 rows x 17]
0 100.0
1 100.0
2 100.0
0 Middle 3 42.9 Rain, Overcast
1 Middle 4 23.7 Rain, Overcast
2 Middle 5 19.0 Rain, Overcast
3 Middle 6 14.3 Overcast
4 Middle 7 47.5 artially cloudy
8 47.5
[5 rows x 9]
10 42.9
11 Middle 11 52.4 artially cloudy
12 Middle 12 47.5 Overcast
13 Middle 13 47.5 Overcast
14 Middle 14 47.5 Overcast
15 Middle 15 38.0 Overcast
Name: Chance Precipitation (%), dtype: float64
```

Pandas Dataframes – Example – Housing Dataset

```
In [24]: df.head()
```

```
Out[24]:
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	\
0	13300000	7420	4	2	3	yes	no	no	
1	12250000	8960	4	4	4	yes	no	no	
2	12250000	9960	3	2	2	yes	no	yes	
3	12215000	7500	4	2	2	yes	no	yes	
4	11410000	7420	4	1	2	yes	yes	yes	

	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	no	yes	2	yes	furnished
1	no	yes	3	no	furnished
2	no	no	2	yes	semi-furnished
3	no	yes	3	yes	furnished
4	no	yes	2	no	furnished

Download at Kaggle: <https://www.kaggle.com/ashydv/housing-dataset>

Pandas Dataframes - Examples

```
import pandas as pd
df = pd.read_csv("Housing.csv")
minimum_bdrms = df["bedrooms"].min()
median_bdrms = df["bedrooms"].median()
maximum_bdrms = df["bedrooms"].max()
print("minimum # of bedrooms:", minimum_bdrms)
print("median # of bedrooms:", median_bdrms)
print("maximum # of bedrooms:", maximum_bdrms)
print("")
print("median values:", df.median().values)
print("")
prices = df["price"]
print("first 5 prices:")
print(prices.head())
print("")
median_price = df["price"].median()
print("median price:", median_price)
print("")
corr_matrix = df.corr()
print("correlation matrix:")
print(corr_matrix["price"].sort_values(ascending=False))
```



```
minimum # of bedrooms: 1
median # of bedrooms: 3.0
maximum # of bedrooms: 6

median values: [4.34e+06 4.60e+03 3.00e+00 1.00e+00 2.00e+00 0.00e+00]

first 5 prices:
0    13300000
1    12250000
2    12250000
3    12215000
4    11410000
Name: price, dtype: int64

median price: 4340000.0

correlation matrix:
price      1.000000
area       0.535997
bathrooms  0.517545
stories    0.420712
parking     0.384394
bedrooms    0.366494
Name: price, dtype: float64
```

Summing-up

- How to work with **Data**
- **NumPy** for scientific computing in Python and examples of **multidimensional arrays**
- **Pandas Dataframes**
- **Implementing agents and environments**

Questions?

A.DiStefano@tees.ac.uk